Zupurb Partner Platform — Statement of Work (SOW)

Document: Partner Roles SOW · **Version:** 1.0 (Draft) · **Date:** 2026-06-22 **Prepared for:** Alan Paul (Zupurb) · **Companion to:** `docs/PARTNER_ROLES_PLAN.md` , `ZUPURB - SOW V6.pdf` **Status:** For review & sign-off

---

1. Background & Purpose

The Zupurb consumer app already surfaces a full **supply side** — venues (`Establishment`), entertainers, vendors, brands, and content creators — but only **venue owners** have a self-service management interface (`zupurb_owner`). This SOW defines the work to deliver **self-service partner interfaces for the remaining roles**, so each partner type can onboard, manage their presence, and transact, while consumers continue to discover and book them in the app.

Per client decision: **a separate application per role**, built **one role at a time**, all running on a **single shared Firebase backend**.

2. Architecture Summary

- **Frontends:** one Next.js web app per partner role, each scaffolded from the existing `zupurb_owner` app. Shared auth/layout/Firebase pieces extracted into a common template/package to limit duplication.
- **Backend:** ONE Firebase project (`zupurb-9580f`) — Firestore, Cloud Functions (callables), and security rules shared by all apps and the consumer app.
- **Creators:** delivered as **in-app** tooling within the Flutter consumer app (influencers are mobile users), not a separate web portal.
- **Reference baseline:** `zupurb_owner` (Venue Owner portal) is already live and is the pattern for all new apps.

3. Roles in Scope

#	Role	Interface	Status entering this SOW
0	Venue / Restaurant Owner	`zupurb_owner` (web)	✅ Exists — baseline, light enhancements only
1	Entertainer (DJ, band, comedian, performer)	`zupurb_entertainer` (web)	New
2	Service Vendor (caterer, photographer, event services)	`zupurb_vendor` (web)	New
3	Brand / Sponsor	`zupurb_brand` (web)	New
4	Influencer / Creator	In-app (Flutter)	New
—	Admin / Staff	Retool / Admin SDK	Out of scope (optional future)

4. Workstreams & Scope

WS0 — Shared Backend Foundation *(prerequisite for all roles)*

Objective: Generalize the existing owner mechanics into a multi-role partner foundation.

In scope:

- Extend `accountType` (`functions/src/lib/schema.ts`) with `entertainer`, `vendor`, `brand`.
- Custom claims `partner: true` + `partnerType` (generalizes `businessOwner`).
- `partners/{uid}` collection `{ partnerType, managedIds[], verified, createdAt }` (generalizes `owners/{uid}`).
- Claim & verification flow extending `claimRequests` + admin approval (`claimOps.ts`) with `entityType`.
- Security-rules helpers `isPartner()` / `isPartnerOf(entityType, entityId)`; gating for `entertainers/`, `vendors/`, `brands/` writes.
- `bookings/{id}` collection + callables `createBooking` (consumer app) and `respondToBooking` (entertainer app), replacing the current mock booking sheet.
- Deployment of schema, rules, indexes, and functions to `zupurb-9580f`.

Deliverables: updated schema & types; deployed Firestore rules + indexes; deployed callables; partner claim/verification flow; data-seeding for test partners. **Acceptance:** a verified partner account of each type can be created, gains the correct claim, and is restricted by rules to only its own entity docs; a consumer booking persists and is retrievable by the entertainer. **Out of scope:** payments/payouts, KYC, multi-currency. **Indicative effort:** 2–3 dev-weeks.

WS1 — Entertainer Partner App (`zupurb_entertainer`) *(build first)*

Objective: Let entertainers manage their profile, availability, and bookings.

In scope (MVP):

- Partner login + role-gating (reuse owner auth pattern).
- Profile editor: name, role/genre, tagline, photos, city, links.
- Availability calendar (set available/blocked dates).
- Booking & inquiry inbox: view, accept, decline, quote.
- Messaging with consumers (reuse existing `entertainer_user` conversation type).
- Reviews: view + respond.
- Dashboard stats: rating, bookings, profile views.

Deliverables: deployed `zupurb_entertainer` web app; wired to WS0 backend; entertainer test account walkthrough. **Acceptance:** an entertainer can edit their public profile (changes reflected in the consumer app), receive a real booking made from the consumer app, and accept/decline/quote it; consumer sees status update. **Out of scope:** payouts, contracts/e-sign, calendar sync (Google/iCal). **Indicative effort:** 3–4 dev-weeks.

WS2 — Vendor Partner App (`zupurb_vendor`)

Objective: Let service vendors manage offerings and inquiries.

In scope (MVP):

- Partner login + role-gating.
- Profile + category (caterer/photographer/etc.), city, tagline, photos.
- Service packages & pricing.
- Inquiry inbox + quotes.
- Reviews: view + respond.
- **Extract shared partner template/package during this app** (auth, layout, Firebase client, UI kit) to stop re-implementing the shell.

Deliverables: deployed `zupurb_vendor` app; reusable `@zupurb/partner-ui` (or equivalent) shared template; vendor test account walkthrough. **Acceptance:** a vendor can publish packages with pricing (visible in consumer app), receive an inquiry, and respond with a quote. **Out of scope:** payments/deposits, portfolio CMS, availability calendar (phase 2). **Indicative effort:** 2–3 dev-weeks (incl. shared-template extraction).

WS3 — Brand Partner App (`zupurb_brand`)

Objective: Let brands run offers and sponsored placements.

In scope (MVP):

- Partner login + role-gating.
- Brand profile (name, category, tagline, assets).
- Run offers/deals — reuse the existing deals engine.
- Sponsored placement in Explore (flagging + basic targeting).
- Basic campaign list & status.

Deliverables: deployed `zupurb_brand` app; offers wired to the deals engine; brand test account walkthrough. **Acceptance:** a brand can create an offer that appears to consumers, activate/deactivate

it, and view basic redemption/engagement counts. **Out of scope:** advanced campaign analytics, ad auctions, billing. **Indicative effort:** 2–3 dev-weeks.

WS4 — Creator Program (in-app, Flutter)

Objective: Give influencers/creators lightweight tooling inside the consumer app.

In scope (MVP):

- Creator Program enrollment + verified-creator badge.
- Post/review analytics (views, saves, follower growth).
- Disclosure tooling (reuse existing disclosure screen).

Deliverables: in-app Creator section; enrollment + analytics; backend support for creator status. **Acceptance:** an eligible user can enroll, receive the verified badge, and view analytics on their posts/reviews. **Out of scope:** brand↔creator marketplace, payouts, promoted posts (phase 2). **Indicative effort:** 2–3 dev-weeks.

WS5 — Cross-cutting

In scope: shared partner template/package (from WS2), per-app CI/CD & hosting, QA pass per app, partner onboarding docs, accessibility & store/web compliance basics. **Indicative effort:** 1–2 dev-weeks (spread across apps).

WS6 — Merchant / Business Revenue Tier (Owner/Business Portal)

Objective: Add merchant-facing monetization to the Business Portal — a paid subscription for enhanced venue profiles, review & demographic insights, and targeted promotion. Applies to the Zupurb Owner Portal and is **reusable for the Yovi Business Portal** (per client feedback).

In scope (MVP):

- **Subscription tiers** (free vs paid) with billing + entitlement gating.
- **Enhanced profile page** for paid venues — richer media, menu, highlights, priority placement (Yelp-style).
- **Review & demographic insights dashboard** — per-category scores plus how they break down by **demographic cohort**; reuses the existing per-cohort / "People Like You" review engine, so it leans on data already captured. Aggregate-only, above a minimum sample size.
- **Targeted promotion** — sponsored placement targeted by **area** (geo) and **demographic cohort** (generalizes the Brand-role sponsored placement).

Deliverables: subscription/billing + entitlement layer; tiered profile UI; insights dashboard; targeted-promotion composer + delivery; merchant test-account walkthrough. **Acceptance:** a merchant can subscribe to a paid tier, unlock the enhanced profile, view cohort-broken-down review insights (above threshold), and run an area + demographic-targeted promotion that reaches the intended audience. **Out of scope:** programmatic ad auctions, external ad networks, self-serve tax/invoicing beyond the billing provider. **Dependencies:** a billing provider (RevenueCat / Stripe); an **ad-policy/legal review for demographic targeting**. **Indicative effort:** 3–4 dev-weeks.

---

5. Deliverables Summary

Workstream	Key deliverable
WS0	Multi-role backend: schema, claims, `partners`, claim flow, rules, bookings + callables (deployed)
WS1	`zupurb_entertainer` web app (profile, availability, bookings, messaging, reviews)
WS2	`zupurb_vendor` web app + shared partner template/package
WS3	`zupurb_brand` web app + offers via deals engine
WS4	In-app Creator Program (enrollment, badge, analytics, disclosure)
WS5	Shared template, CI/CD per app, QA, onboarding docs
WS6	Merchant revenue tier: subscription + enhanced profiles + review/demographic insights + area/demographic targeted promotion

6. Milestones, Sequencing & Timeline

Sequential per client decision. Indicative for **one full-stack engineer**; parallelization shortens calendar time.

Milestone	Scope	Indicative duration
M1	WS0 — Backend foundation deployed	Weeks 1–3
M2	WS1 — Entertainer app live + real bookings	Weeks 4–7
M3	WS2 — Vendor app + shared template	Weeks 8–10
M4	WS3 — Brand app + offers	Weeks 11–13
M5	WS4 — Creator Program (in-app)	Weeks 14–16
M6	WS5 — Hardening, QA, onboarding docs	Weeks 16–17
M7	WS6 — Merchant revenue tier (subscription, insights, targeted promotion)	Weeks 18–21

Total indicative: ~20–22 calendar weeks (single engineer).

7. Effort & Indicative Pricing

Workstream	Indicative effort (dev-weeks)
WS0 — Backend foundation	2–3
WS1 — Entertainer	3–4
WS2 — Vendor (+ template)	2–3
WS3 — Brand	2–3
WS4 — Creator Program	2–3
WS5 — Cross-cutting	1–2
WS6 — Merchant revenue tier	3–4
Total	**15–22 dev-weeks**

> Pricing = effort × agreed rate card (to be supplied). This SOW quantifies effort, not currency.

8. Assumptions

- Reuse of the existing `zupurb_owner` app and its auth/role patterns is permitted.
- One shared Firebase project (`zupurb-9580f`); no per-role project split.
- MVP scope excludes money movement (payouts, deposits, billing) unless added via change control.
- Designs follow the existing Zupurb design system; net-new screens use it without a separate design phase.
- Test/demo partner data can be seeded.

9. Dependencies

- 🔴 **Deploy access to `zupurb-9580f`** — schema, rules, claims, and callables must be deployable. Currently blocked (the CLI account lacks permission); must be resolved before WS0 ships.
- App Check configuration for web (callables currently fail on web without it) — required for callable-backed flows on the web partner apps.
- Any third-party services for later phases (payments, calendar sync, e-sign) procured separately.
- Hosting targets for the new web apps (e.g., Firebase Hosting / Vercel).

10. Out of Scope (global, unless added via change control)

- Native mobile partner apps (partner interfaces are web, except in-app Creator tooling).
- Payments, payouts, deposits, escrow, invoicing, tax.
- KYC/identity verification, contracts/e-signature.

- Admin/staff console (remains on Retool/Admin SDK).
- Advanced analytics/ad auction/recommendation tuning beyond stated MVP.

11. Roles & Responsibilities

Area	Client (Zupurb)	Delivery
Firebase project ownership & deploy access	✅ Provide	—
Product decisions, scope sign-off	✅	Advise
Design assets / brand	✅ Provide	Apply
Backend, rules, callables	—	✅ Build
Partner web apps & in-app creator	—	✅ Build
QA & acceptance testing	✅ Participate	✅ Run
Third-party accounts (payments, etc.)	✅ Provide	Integrate

12. Acceptance & Sign-off

Each milestone is accepted when its workstream **acceptance criteria** (§4) are demonstrated on `zupurb-9580f` with a test partner account, and the client signs off. Defects found during acceptance are remediated within the milestone; new requests are handled via change control.

13. Change Control

Scope changes (new features, money movement, native apps, added analytics) are estimated as a written change request (scope + effort delta) and appended to this SOW upon mutual agreement.

Prepared as a companion to the Partner Roles Plan. Effort and timeline are indicative and refine once WS0 backend access is confirmed and designs are final.